

the key during the encryption or decryption process. Line 5 starts a loop that iterates over each character in the data string. The `enumerate()` function is used to get both the index and the character in each iteration. Line 6 calculates the shift amount for the current character based on the corresponding character in the key. The code `alphabet.index(key[i % key_length])` gives the position of the key's character in the alphabet. The modulus operation (`i % key_length`) ensures that the key repeats if it is shorter than the data. Finally, `mode` is multiplied by the shift value. If `mode` is 1, the character will be shifted forward (encryption). If `mode` is -1, the character will be shifted backward (decryption). Line 7 calculates the new character after shifting. The code `alphabet.index(char)` gives the position of the current character in the alphabet. The shift value is added to this index for encryption or subtracted for decryption. The modulus operation (`% 26`) ensures that the index wraps around within the alphabet if it exceeds 25. Then, the index of the string `alphabet` retrieves the character corresponding to the new shifted position. Line 8 appends the newly calculated character in `new_char` to the list `text`, building the result step by step. Finally, after all characters have been processed, Line 9 joins the list `text` into a single string and returns the result, either the fully encrypted or decrypted text.

Now, we need to test this Vigenère cipher function using a plaintext and a key. The following lines of code are added to the end of the function `vigenere_cipher()` to perform this testing (line numbers are included for clarity).

```
10. plaintext = "CRYPTOGRAPHY"
11. key = "BCDE"
12. ciphertext = vigenere_cipher(plaintext, key, 1)
13. print("Encrypted:", ciphertext)
14. decrypted_text = vigenere_cipher(ciphertext, key, -1)
15. print("Decrypted:", decrypted_text)
```

This code for testing the function `vigenere_cipher()` is very similar to the code used to test the function `caesar_cipher()`. The main difference is that, instead of using a fixed shift, the Vigenère cipher uses a key (Line 14). As mentioned earlier, a `mode` value of 1 is used for encryption, and a `mode` value of -1 is used for decryption.

Now, let us execute the program 'vigenere21.sagews'. Upon execution, the following output will be displayed on the screen.

```
Encrypted: DTBTUQJVBKRC
Decrypted: CRYPTOGRAPHY
```

The output once again confirms that the plaintext and ciphertext are in no way related, and it also demonstrates that encryption with the Vigenère cipher is reversible.

Now, try the key 'MXQZVAIXOZHQ' by changing the code in Line 17 to `key = "MXQZVAIXOZHQ"`. Upon executing the modified code, the encrypted text obtained is 'OOOOOOOOOOOOO'. Funny, isn't it? From the output, it can also be verified that the correct plaintext is obtained upon decryption.

The security of the Vigenère cipher relies heavily on the secrecy of the key. If the key is compromised, the cipher can be easily broken. For over three centuries, the Vigenère cipher was believed to be unbreakable. The French even referred to it as 'le chiffage indéchiffrable' (the indecipherable cipher). However, in 1863, Friedrich Kasiski, a military officer from Prussia (a German-speaking nation that later became a key part of Germany), published a method for breaking the Vigenère cipher by analysing repeated sequences in the ciphertext. This technique, known as the 'Kasiski examination', allowed cryptanalysts (people who study and break codes and ciphers to uncover hidden information) to deduce the key length by exploiting patterns in the encrypted message, marking a major breakthrough in cryptography.

Just eight years later, in 1871, Prussia defeated France in the Franco-Prussian War, leading to the formation of the German Empire. While it's uncertain whether the breaking of French military Vigenère cipher codes directly contributed to their defeat, Kasiski's method undeniably weakened the perception of the cipher's invincibility. Could the cipher's compromise have played a part in France's downfall? Perhaps.

This article provided a gentle introduction to the terminology used in cybersecurity and related areas. We also explored the implementation details of two important classical encryption schemes, albeit in a simplified manner, and implemented them using SageMath. Notably, the code we discussed is predominantly Python, once again highlighting SageMath's ability to extend Python's capabilities.

In the next article in this SageMath series, we will continue our exploration of classical encryption schemes by examining transposition ciphers, which scramble the positions of characters rather than altering the characters themselves. We will also begin our discussion on symmetric key cryptography, also known as private key cryptography or secret key cryptography, along with its implementation in SageMath. 

 By: Dr Deepu Benson

The author is assistant professor at Chinmaya Vishwa Vidyapeeth (deemed to be university), Ernakulam, Kerala. He is a free software enthusiast, and is interested in theoretical computer science.